Laboratorio NRF24L01.

Paula Quintero y Ediem Valero.
Comunicación Digital.
Universidad Militar Nueva Granada
Bogotá, Colombia
Octubre 14, 2026.

Resumen - En esta práctica se diseña un sistema de comunicación digital inalámbrica empleando dispositivos Raspberry Pi Pico/Pico 2W y módulos NRF24L01, con el propósito de estudiar el funcionamiento de un enlace inalámbrico y analizar diferentes características de su comunicación. El sistema permite configurar parámetros como el canal de operación, la velocidad de transmisión y la potencia, así como seleccionar y transmitir diferentes tipos de mensajes mediante una interfaz conformada por pulsadores, pantallas OLED y LEDs. Luego, se analiza la comunicación entre el microcontrolador y el transceptor mediante el protocolo SPI, identificando los comandos, bytes transmitidos, señales de control y tiempos asociados a diferentes operaciones. Además, se estudia el alcance del sistema para diferentes condiciones de potencia y sensibilidad, se analiza el comportamiento de los errores durante la transmisión de información y se realizan mediciones mediante un analizador de espectros para verificar la frecuencia de operación, el ancho de banda y el comportamiento espectral de la comunicación. Finalmente, se analiza la canalización en la banda de 2,4 GHz y su relación con otras comunicaciones inalámbricas, permitiendo integrar los aspectos de configuración, transmisión, medición y análisis de un sistema de comunicación digital inalámbrico.

Palabras clave: NRF24L01, Raspberry Pi Pico, SPI, radiofrecuencia, alcance inalámbrico, tasa de error de bits, analizador lógico, analizador de espectros.

I. INTRODUCCIÓN.

La comunicación digital inalámbrica constituye un área fundamental de las telecomunicaciones, debido a su aplicación en sistemas que requieren el intercambio de información sin necesidad de establecer una conexión física entre los dispositivos. Dentro de este contexto, los sistemas de radiofrecuencia permiten estudiar aspectos relacionados con la transmisión de datos, la configuración de canales, la velocidad de comunicación, la potencia de transmisión y el comportamiento de las señales en el medio inalámbrico. La práctica propuesta aborda estos conceptos mediante el desarrollo de un sistema de comunicación basado en dispositivos Raspberry Pi Pico/Pico 2W y módulos transceptores NRF24L01, los cuales emplean modulación GFSK y permiten establecer enlaces inalámbricos en la banda de 2,4 GHz.

El propósito de esta práctica es diseñar e implementar un sistema funcional de comunicación digital inalámbrica que permita configurar diferentes parámetros de operación y transmitir información entre dos dispositivos. Para ello, el sistema incorpora elementos de interacción como pulsadores, pantallas OLED y LEDs, mediante los cuales es posible seleccionar parámetros como el canal de operación, la velocidad de transmisión y la potencia configurada. De esta manera, además de comprobar el funcionamiento del enlace inalámbrico, se busca relacionar la configuración realizada en el microcontrolador con el comportamiento observado en el transceptor y en la señal transmitida.

La importancia de este análisis radica en que el funcionamiento de un sistema inalámbrico no depende únicamente de establecer la comunicación entre dos dispositivos, sino también de comprender cómo se configuran y transmiten los datos a través de las diferentes capas involucradas. Por esta razón, la práctica contempla el análisis de la comunicación SPI entre el Raspberry Pi y el NRF24L01, identificando los comandos empleados, los bytes transmitidos, las señales de control y los tiempos asociados a las transferencias. Este análisis permite observar directamente cómo el microcontrolador configura y controla el transceptor antes y durante el proceso de transmisión.

Asimismo, la práctica aborda diferentes aspectos relacionados con el desempeño del enlace inalámbrico. Se plantea el estudio del alcance teórico y experimental para diferentes condiciones de potencia y sensibilidad, así como el análisis del comportamiento de los errores durante la transmisión de información mediante archivos de texto y datos pseudoaleatorios. Finalmente, se contempla la caracterización de la señal mediante un analizador de espectros, con el propósito de verificar la frecuencia de operación, determinar el ancho de banda y analizar la distribución de canales dentro de la banda de 2,4 GHz, incluyendo su posible interacción con otras tecnologías inalámbricas.

En este sentido, el problema abordado consiste en implementar y caracterizar un enlace de comunicación digital inalámbrica, relacionando la configuración del sistema con su comportamiento real tanto en el dominio de los datos como en el de radiofrecuencia. El enfoque empleado combina la implementación práctica del sistema con herramientas de medición y análisis, como el analizador lógico y el analizador de espectros, permitiendo estudiar de manera integral el proceso de comunicación.

Este informe documenta el desarrollo de la práctica, comenzando con los fundamentos relacionados con la comunicación inalámbrica, el módulo NRF24L01 y el protocolo SPI. Luego, se presenta la implementación del

sistema y el análisis de las diferentes pruebas planteadas en la guía, incluyendo el estudio de la interfaz SPI, el alcance del enlace, los errores de transmisión y el comportamiento espectral de la señal. Finalmente, se presentan y analizan los resultados obtenidos para establecer las principales conclusiones sobre el funcionamiento y desempeño del sistema implementado.

II. OBJETIVOS

OBJETIVO GENERAL:

Diseñar e implementar un sistema de comunicación digital inalámbrico basado en modulación GFSK, utilizando el módulo NRF24L01 y un Raspberry Pi Pico/Pico 2W.

OBJETIVOS ESPECÍFICOS:

- Analizar los detalles de la capa física del estándar SPI.
- Estimar teórica y experimentalmente el alcance del sistema inalámbrico implementado.
- Desarrollar habilidades para la medición de señales inalámbricas mediante el uso de un analizador de espectros.

III. MARCO TEÓRICO

1. *Comunicación digital inalámbrica:*

La comunicación digital inalámbrica permite transmitir información entre diferentes dispositivos utilizando ondas electromagnéticas, sin necesidad de una conexión física entre el transmisor y el receptor. En este tipo de sistemas, la información digital se convierte en una señal que puede ser transmitida por radiofrecuencia y posteriormente recuperada por el dispositivo receptor. [1]

En esta práctica se implementa un sistema de comunicación inalámbrica utilizando un Raspberry Pi Pico/Pico 2W y un módulo NRF24L01. El sistema trabaja en la banda de 2,4 GHz y permite configurar diferentes parámetros de transmisión, como el canal, la velocidad de transmisión y la potencia. La guía también plantea el uso de mensajes de diferentes longitudes para comprobar el funcionamiento de la comunicación.

2. *Modulación GFSK:*

GFSK (Gaussian Frequency Shift Keying) es una técnica de modulación digital en la que la información se representa mediante cambios en la frecuencia de una señal portadora. Antes de realizar la modulación, los cambios de la señal digital pasan por un filtro gaussiano, lo que permite suavizar las transiciones de frecuencia y reducir el ancho de banda ocupado por la señal.

Esta modulación es utilizada por el NRF24L01 para realizar la transmisión inalámbrica. Una de sus ventajas es que permite realizar comunicaciones digitales utilizando un ancho de banda relativamente reducido, lo cual es importante cuando existen varios sistemas trabajando en una misma banda de frecuencia. [2]

3. *Modulo NRF_24L01:*

El NRF24L01 es un transceptor inalámbrico que permite realizar transmisión y recepción de datos en la banda de 2,4 GHz. Al ser un transceptor, el mismo dispositivo puede utilizarse tanto para enviar como para recibir información. El módulo puede configurarse mediante diferentes parámetros, entre ellos el canal de radiofrecuencia, la velocidad de transmisión y la potencia de salida. En la práctica, estos parámetros son controlados desde el Raspberry Pi Pico y pueden ser modificados mediante los elementos de interacción implementados en el sistema. [3]

El canal de operación determina la frecuencia utilizada para la comunicación. De forma general, para el NRF24L01 la frecuencia se relaciona con el número de canal mediante:

$$f_{RF} = 2400 + RF_{CH}$$

donde f_{RF} corresponde a la frecuencia de operación en MHz y RF_{CH} corresponde al número de canal configurado.

4. *Raspberry Pi Pico y control del transceptor:*

El Raspberry Pi Pico/Pico 2W funciona como el dispositivo encargado de controlar el módulo NRF24L01. Por medio de sus pines de entrada y salida y de la interfaz SPI, el microcontrolador puede configurar los registros del transceptor y enviar los datos que posteriormente serán transmitidos de forma inalámbrica.

Además, el sistema desarrollado utiliza elementos de interacción como pulsadores, LEDs y una pantalla OLED. Estos componentes permiten seleccionar diferentes opciones de configuración y mostrar al usuario información relacionada con el funcionamiento del sistema, como el canal, la velocidad y la potencia configurada. Estos elementos hacen parte de los requerimientos establecidos para la implementación de la práctica.

5. *Protocolo SPI:*

SPI (Serial Peripheral Interface) es un protocolo de comunicación serial utilizado para intercambiar información entre dispositivos electrónicos. En este caso, se utiliza para establecer la comunicación entre el Raspberry Pi Pico y el NRF24L01.

La comunicación SPI utiliza principalmente cuatro señales:

- SCK: señal de reloj generada por el dispositivo maestro.
- MOSI: línea por la cual el maestro envía información al dispositivo esclavo.
- MISO: línea por la cual el esclavo envía información al maestro.
- CSN: señal utilizada para seleccionar el dispositivo con el que se desea realizar la comunicación.

En el sistema implementado, el Raspberry Pi Pico funciona como maestro y el NRF24L01 como dispositivo controlado mediante SPI. A través de esta interfaz se pueden realizar operaciones como lectura y escritura de registros, configuración del módulo y transferencia de datos. [4]

6. *Alcance de una comunicación inalámbrica:*

El alcance de un sistema inalámbrico corresponde a la distancia máxima a la cual el receptor puede recibir correctamente la información transmitida. Este alcance

depende de diferentes factores, entre ellos la potencia de transmisión, la sensibilidad del receptor, la frecuencia de operación, las características de las antenas y las condiciones del entorno.
Una mayor potencia de transmisión puede permitir que la señal alcance una distancia mayor. Sin embargo, esto no significa que el alcance dependa únicamente de la potencia, ya que también existen pérdidas durante la propagación de la señal y características propias del receptor. [5]

7. Analizador lógico:

Un analizador lógico es un instrumento utilizado para capturar y visualizar señales digitales en función del tiempo. Permite observar las transiciones de las señales y, mediante herramientas de decodificación, interpretar protocolos de comunicación.
En esta práctica se utiliza para analizar la comunicación SPI entre el Raspberry Pi Pico y el NRF24L01. Con las capturas realizadas es posible identificar cuándo se activa la señal CSN, observar los pulsos de reloj y determinar los bytes enviados y recibidos durante diferentes operaciones. Este análisis permite relacionar lo que ocurre en el código del microcontrolador con las señales eléctricas que realmente aparecen en la interfaz SPI. [6]

8. Analizador de espectros:

El analizador de espectros es un instrumento utilizado para observar cómo se distribuye la potencia de una señal en función de la frecuencia. A diferencia de un analizador lógico, que permite estudiar principalmente señales digitales en el dominio del tiempo, el analizador de espectros permite estudiar las características de una señal de radiofrecuencia. [7]
En esta práctica se utiliza para comprobar la frecuencia en la que está operando el NRF24L01 y analizar el ancho de banda ocupado por la comunicación. También permite observar la ubicación de la señal dentro de la banda de 2,4 GHz y estudiar la posible interacción entre canales cercanos.
La guía establece como parte del análisis de espectro la validación de la frecuencia del canal, la determinación del ancho de banda, el análisis de la asignación de canales y el uso de la función de espectrograma del analizador.

IV. PROCEDIMIENTO

1. Implementación del sistema inalámbrico:

Para implementar el sistema de comunicación inalámbrica se utilizaron dos placas Raspberry Pi Pico/Pico 2W, cada una conectada a un módulo transceptor NRF24L01 con antena y una pantalla OLED. Ambos montajes se realizaron siguiendo la misma distribución de pines, con el propósito de facilitar la programación y mantener una configuración equivalente entre los dos nodos de comunicación.
El módulo NRF24L01 se conectó a la Raspberry Pi mediante la interfaz SPI, utilizada para configurar el transceptor y enviar los datos que posteriormente se transmiten de forma inalámbrica. Por su parte, la pantalla OLED se conectó mediante la interfaz I²C, permitiendo mostrar información sobre la configuración y el funcionamiento del sistema.

- *Conexión del módulo NRF24L01:*

El módulo NRF24L01 se conectó a los pines de alimentación y comunicación de la Raspberry Pi Pico. La distribución de las conexiones se presenta en la tabla 1.

Pin del NRF24L01	Pin de la Raspberry Pi Pico	Función
VCC	3V3(OUT)	Alimentación de 3,3V
GND	GND	Tierra común
CE	GP17	Habilitación del transceptor
CSN	GP20	Selección del dispositivo SPI
SCK	GP18	Señal de reloj SPI
MOSI	GP19	Datos enviados desde la Raspberry Pi al módulo
MISO	GP16	Datos enviados desde el módulo a la Raspberry Pi

Tabla 1. Conexión del NRF24L01 con la Raspberry Pi Pico.

Cabe aclarar que el pin de alimentación del NRF24L01 se conectó a 3V3(OUT) de la Raspberry Pi Pico, ya que el módulo trabaja con una alimentación de 3,3 V. Las señales CE y CSN se utilizaron para controlar el funcionamiento y la selección del transceptor, mientras que los pines SCK, MOSI y MISO permitieron establecer la comunicación mediante SPI.

- *Conexión de la pantalla OLED:*

La pantalla OLED se conectó a la Raspberry Pi Pico mediante el protocolo I²C, utilizando dos pines para la comunicación y las conexiones correspondientes para alimentación y tierra. La distribución de las conexiones se presenta en la tabla 2.

Pin de la pantalla OLED	Pin de la Raspberry Pi Pico	Función
VCC	3V3 (OUT)	Alimentación de la pantalla
GND	GND	Tierra común
SCL	GP5	Señal de reloj I²C
SDA	GP4	Línea de transmisión de datos I²C

Tabla 2. Conexión de la pantalla OLED con la Raspberry Pi Pico.

La línea SCL proporciona la señal de reloj que sincroniza la comunicación, mientras que SDA permite intercambiar los datos entre la Raspberry Pi y la pantalla. La pantalla se configuró para trabajar a una frecuencia de comunicación I²C de 400 kHz y con la dirección 0x3C.

- *Conexión de los elementos de control e indicación:*

Además de los módulos de comunicación y las pantallas, se incorporaron dos pulsadores y un LED para facilitar la interacción con el sistema. La distribución de las conexiones se presenta en la tabla 3.

Elemento	Pin de la Raspberry Pi Pico	Función
Pulsador 1	GP10	Selección del parámetro
Pulsador 2	GP11	Modificación del valor seleccionado
LED	GP12	Indicador visual del funcionamiento

Tabla 3. Conexión de los pulsadores a la Raspberry Pi Pico.

Los pulsadores se configuraron como entradas digitales con resistencias internas de pull-up, de manera que el programa pudiera detectar las pulsaciones realizadas por el usuario. El LED se utilizó como indicador visual de las acciones del sistema, de acuerdo con la programación implementada.

En la figura 1 se muestra el montaje realizado en las dos Raspberry Pi Pico, conservando la misma asignación de pines para el NRF24L01, la pantalla OLED, los pulsadores y el LED.

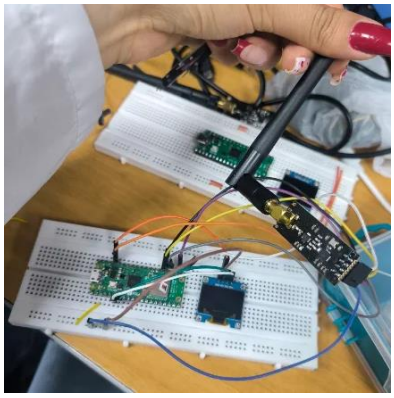


Figura 1. Montaje realizado para el laboratorio.

De esta manera, cada nodo quedó conformado por una Raspberry Pi Pico, un módulo NRF24L01 con antena, una pantalla OLED y los elementos de control. Esta configuración permitió disponer de dos dispositivos con la misma estructura de hardware para desarrollar las pruebas de comunicación inalámbrica.

2. Configuración del módulo NRF24L01:

Después de realizar el montaje físico, se programó la Raspberry Pi Pico mediante MicroPython utilizando el entorno de desarrollo Thonny. El código se encargó de configurar el módulo NRF24L01 a través de la interfaz SPI, establecer los parámetros de comunicación y permitir que el usuario modificara las opciones del sistema mediante los pulsadores.
Inicialmente, se definieron los pines de comunicación SPI, la señal de habilitación del módulo (CE) y la señal de selección (CSN). También se configuró la pantalla OLED mediante la interfaz I²C, con el propósito de mostrar los parámetros seleccionados durante la ejecución del programa.

2.1. Configuración de los parámetros de comunicación:

En el código se establecieron valores iniciales para los parámetros de funcionamiento del NRF24L01. Estos valores se almacenaron en variables y listas para facilitar su modificación desde la interfaz del sistema.
Los parámetros configurados se muestran en la tabla 4:

Parámetro	Opciones disponibles
Canal	Canales del 70 al 80
Velocidad de transmisión	250 kbps, 1 Mbps y 2 Mbps
Potencia de transmisión	−18 dBm, −12 dBm, −6 dBm y 0 dBm
Mensaje	EDIEM VALERO, PAULA QUINTERO, ALFABETO y ALFABETO NUMERICO
Modo de operación	Transmisión (TX) y recepción (RX)
Tipo de envío	UNICO y LOOP

Tabla 4.Parámetros configurados en la NRF24L01.

Estos parámetros se utilizaron para controlar la configuración del transceptor y seleccionar el comportamiento del sistema durante las pruebas.

2.2. Configuración mediante SPI:

Para establecer la comunicación con el NRF24L01 se utilizó la interfaz SPI de la Raspberry Pi Pico, configurada con una frecuencia de reloj de 1 MHz, polaridad del reloj igual a cero y fase igual a cero.
Mediante esta interfaz se implementaron funciones para realizar operaciones de lectura y escritura de registros. Estas funciones permiten enviar los comandos correspondientes al transceptor y transferir los valores necesarios para configurar su funcionamiento.
El programa utilizó diferentes registros del NRF24L01, entre ellos:

- CONFIG: permite configurar el estado de alimentación y seleccionar el modo de transmisión o recepción.
- RF_CH: permite establecer el canal de radiofrecuencia.
- RF_SETUP: permite configurar la velocidad de transmisión y la potencia de salida.
- EN_AA: permite habilitar o deshabilitar las confirmaciones automáticas de recepción.
- TX_ADDR y RX_ADDR_P0: permiten establecer las direcciones utilizadas para la comunicación.
- STATUS: permite gestionar las banderas de estado del transceptor.

Durante la inicialización, el programa deshabilitó las confirmaciones automáticas, configuró la dirección de comunicación, estableció el canal y ajustó la velocidad y la potencia seleccionadas. También se limpiaron las colas de transmisión y recepción antes de continuar con la configuración del modo de funcionamiento.

2.3. Selección de velocidad y potencia:

La velocidad y la potencia se configuraron mediante la función *configurar_rf_setup()*. Esta función identifica las opciones seleccionadas por el usuario y asigna los bits correspondientes del registro RF_SETUP.
Para modificar estos parámetros, el programa utiliza índices que permiten recorrer las opciones disponibles. Cuando el usuario selecciona una nueva velocidad o potencia, se actualiza el índice correspondiente y se vuelve a configurar el módulo mediante SPI.
De esta manera, no fue necesario modificar manualmente el código cada vez que se quería probar una configuración diferente, ya que los cambios podían realizarse desde los pulsadores del sistema.

2.4. Control mediante pulsadores y pantalla OLED:

Para facilitar el manejo del sistema se utilizaron dos pulsadores conectados a los pines GP10 y GP11 de la Raspberry Pi Pico.
El primer pulsador permite recorrer los parámetros disponibles: canal, velocidad, potencia, mensaje, modo TX/RX y tipo de envío UNICO/LOOP. El segundo pulsador permite modificar el valor del parámetro seleccionado.
Cuando se detecta una pulsación, el programa actualiza la variable correspondiente, ejecuta nuevamente la configuración del NRF24L01 y actualiza la información mostrada en la pantalla OLED.
La pantalla permite visualizar los parámetros de funcionamiento y facilita la interacción con el sistema sin necesidad de modificar directamente las variables desde el código. Además, se incorporó un LED conectado al pin GP12 para indicar visualmente diferentes estados de funcionamiento.

2.5. Configuración de la transmisión de mensajes:

Para transmitir información, el programa utiliza una función denominada transmitir(), encargada de convertir el mensaje seleccionado en una secuencia de bytes.
Debido a que el NRF24L01 admite cargas útiles de hasta 32 bytes por paquete, el programa verifica la longitud del mensaje. Si supera ese límite, lo recorta; si es más corto, completa el contenido con bytes de valor cero hasta alcanzar los 32 bytes.
Posteriormente, se limpia la cola de transmisión, se envía el comando W_TX_PAYLOAD y se transfieren los bytes del mensaje mediante SPI. Finalmente, se genera un pulso en la señal CE para iniciar el proceso de transmisión.
El programa contempla dos tipos de envío. En el modo LOOP, el mensaje se transmite repetidamente con una pausa de 1 ms entre iteraciones y el LED cambia de estado aproximadamente cada 100 ms. En el modo UNICO, el programa evita la transmisión repetitiva automática mientras permanece en esa opción. Este comportamiento corresponde a la lógica implementada en el código utilizado.

2.6. Pseudocódigo del programa:

El siguiente pseudocódigo resume el funcionamiento general del sistema implementado:

```
INICIO

    Configurar pantalla OLED mediante I2C
    Configurar pulsadores y LED
    Configurar interfaz SPI
    Definir registros y comandos del NRF24L01

    Establecer parámetros iniciales:
        Canal = 76
        Velocidad = 1 Mbps
        Potencia = 0 dBm
        Mensaje = "EDIEM VALERO"
        Modo = TX
        Tipo de envío = LOOP

    Configurar el módulo NRF24L01
    Mostrar los parámetros en la pantalla OLED

    MIENTRAS el programa esté en ejecución:

        Leer el estado de los pulsadores

        SI se presiona el pulsador 1:
            Seleccionar el siguiente parámetro
            Actualizar la pantalla OLED

        SI se presiona el pulsador 2:
            Cambiar el valor del parámetro seleccionado
            Actualizar la configuración del NRF24L01
            Actualizar la pantalla OLED

        SI el modo seleccionado es TX:

            SI el tipo de envío es LOOP:
                Convertir el mensaje en bytes
                Preparar la carga útil de 32 bytes
                Enviar el comando de transmisión por SPI
                Transferir los datos al NRF24L01
                Activar CE para iniciar la transmisión
                Controlar el parpadeo del LED
                Esperar 1 ms

            SI el tipo de envío es UNICO:
                No repetir automáticamente la transmisión
                Mantener el LED apagado

        SI el modo seleccionado es RX:
            Activar el estado de recepción
            Controlar CE
            Mantener el LED encendido

FIN
```

3. Prueba de transmisión inalámbrica:

Una vez configurados los módulos NRF24L01 y los elementos de interacción, se preparó el sistema para realizar pruebas de transmisión inalámbrica entre los dos dispositivos. Para ello, se utilizó una Raspberry Pi Pico como transmisor y otra como receptor, ambas conectadas a sus respectivos módulos NRF24L01 con antena.
Inicialmente, se verificó que cada dispositivo tuviera cargado el programa correspondiente y que los módulos estuvieran conectados de acuerdo con la distribución de pines establecida. También se revisó que los parámetros de comunicación fueran compatibles entre ambos dispositivos, especialmente el canal de radiofrecuencia, la velocidad de transmisión y las direcciones utilizadas para la comunicación.

3.1. Preparación del transmisor:

En el dispositivo transmisor se seleccionó el modo TX mediante la interfaz de usuario. A continuación, se configuraron los parámetros de comunicación utilizando los pulsadores y la pantalla OLED.
El programa permitió seleccionar diferentes mensajes para realizar las pruebas, entre ellos nombres y textos relacionados con alfabetos. Una vez seleccionado el mensaje, la función de transmisión lo convirtió en una secuencia de bytes y preparó la carga útil para enviarla al NRF24L01 mediante SPI.
Posteriormente, el programa activó la señal CE durante el tiempo establecido para iniciar la transmisión del paquete. Este proceso se realizó de acuerdo con el tipo de envío seleccionado en la interfaz: LOOP o UNICO.

3.2. Preparación del receptor:

En la segunda Raspberry Pi Pico se configuró el módulo NRF24L01 para trabajar en modo RX. Para permitir la comunicación, se establecieron los parámetros correspondientes al canal, la velocidad de transmisión y la dirección de recepción, de manera que coincidieran con los utilizados por el transmisor.

También se configuraron los pines de control del módulo y se preparó el programa para mantener el dispositivo en estado de recepción. De esta manera, el receptor quedó dispuesto para recibir los paquetes enviados desde el otro nodo.

3.3. Ejecución de la prueba:

Con ambos dispositivos configurados, se inició la transmisión de los mensajes desde el primer nodo hacia el segundo. Durante esta etapa se utilizó la interfaz de usuario para seleccionar los mensajes y controlar los parámetros disponibles.

El procedimiento se planteó para utilizar tanto mensajes cortos como mensajes de mayor longitud, con el propósito de disponer de diferentes tipos de información para las pruebas posteriores. Asimismo, se contempló el funcionamiento del sistema en los modos de transmisión y recepción.

Esta prueba permitió preparar el enlace inalámbrico para los análisis posteriores de la práctica, incluyendo la observación de la señal mediante el analizador de espectros, la evaluación del alcance y el estudio de los errores de transmisión.

4. Análisis de la comunicación SPI:

El análisis de la comunicación SPI se realizó utilizando un analizador lógico conectado a la Raspberry Pi Pico que controlaba el módulo NRF24L01. El propósito fue observar las señales digitales de la interfaz y decodificar las transferencias realizadas entre el microcontrolador y el transceptor.

4.1. Conexión y configuración del analizador lógico:

Se conectaron las entradas del analizador lógico a las señales utilizadas por el protocolo SPI, tomando como referencia la tierra común del circuito. La distribución de las conexiones se presenta en la tabla 5.

Canal del analizador lógico	Señal	Pin de la Raspberry Pi Pico
D0	SCK	GP18
D1	MOSI	GP19
D2	MISO	GP16
D3	CSN	GP20
D4	CE	GP17
GND	Tierra común	GND

Tabla 5. Distribución de las conexiones para el analizador lógico.

En la figura 2 se observan las conexiones realizadas en el montaje.

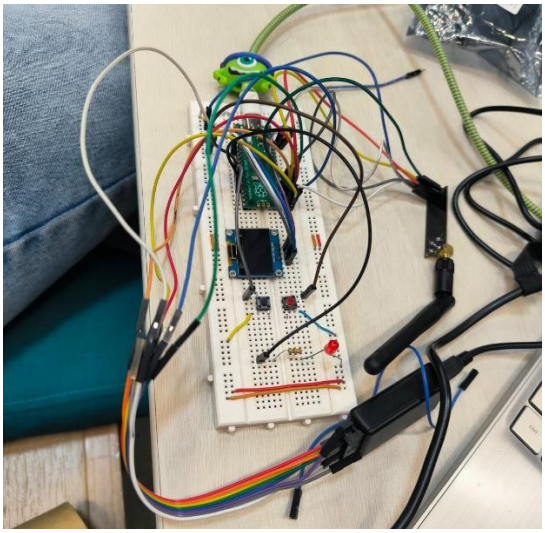


Figura 2. Conexión del analizador lógico al montaje inicial.

Luego, en el software Saleae Logic 2 se configuró la captura de las señales digitales y se habilitó el decodificador SPI. Para realizar la decodificación se configuraron los parámetros que se observan en la figura 3.

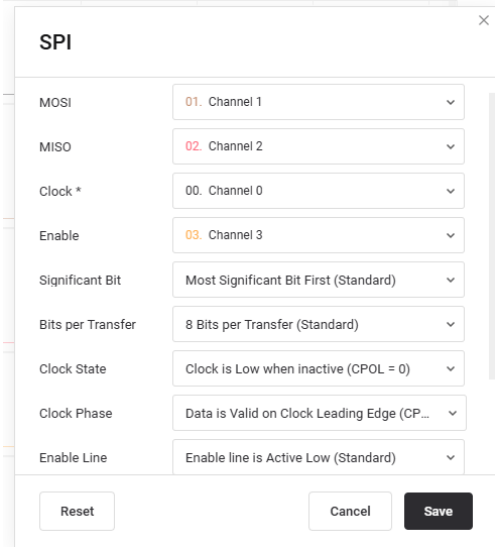


Figura 3. Parámetros configurados en Logic 2.

También se ajustó la frecuencia de muestreo en 12Ms/s para obtener capturas que permitieran observar las transferencias SPI y decodificar los bytes enviados y recibidos.

4.2. Lectura de un registro del NRF24L01:

Para la primera prueba se realizó una operación de lectura de un registro interno del NRF24L01. Se seleccionó el registro RF_CH, encargado de almacenar el número del canal de radiofrecuencia configurado en el transceptor.

El procedimiento consistió en iniciar una captura con el analizador lógico y ejecutar desde la Raspberry Pi Pico la función encargada de leer el registro. Durante esta operación, el microcontrolador seleccionó el NRF24L01 mediante la señal CSN y envió por SPI el comando de lectura correspondiente.

A continuación, se transmitió un byte adicional para solicitar el contenido del registro. Mientras se realizaba esta operación, se observaron las señales MOSI y MISO para identificar la información enviada por el microcontrolador y la respuesta del transceptor. Finalmente, se desactivó la selección del

dispositivo y se detuvo la captura para examinar la transferencia mediante el decodificador SPI.

4.3. Escritura del canal de radiofrecuencia:

En la segunda prueba se realizó una operación de escritura sobre el registro RF_CH, con el propósito de modificar el canal de operación del NRF24L01.

Para esto, se inició una nueva captura en el analizador lógico y se ejecutó desde el programa la función encargada de escribir en un registro. El microcontrolador activó la señal CSN y envió el comando de escritura junto con el valor correspondiente al canal que se deseaba configurar.

Durante la transferencia se observaron las señales de reloj, datos y selección del dispositivo. Una vez finalizado el envío, se desactivó CSN y se terminó la captura.

Luego, se realizó una nueva operación de lectura sobre el mismo registro para comprobar la configuración mediante la interfaz SPI. Esta verificación permitió disponer de una captura independiente para analizar la operación de escritura y la lectura posterior.

4.4. Transferencia de un paquete al transceptor:

Para la tercera prueba se analizó el procedimiento mediante el cual la Raspberry Pi Pico carga un mensaje en el NRF24L01 antes de iniciar su transmisión inalámbrica.

Inicialmente, se preparó el mensaje seleccionado en el programa y se convirtió su contenido a una secuencia de bytes. Después, se inició la captura con el analizador lógico y se ejecutó la función de transmisión.

El procedimiento incluyó las siguientes operaciones:

- Limpiar la cola de transmisión del NRF24L01 mediante el comando correspondiente.
- Activar la señal CSN para seleccionar el transceptor.
- Enviar el comando W_TX_PAYLOAD para indicar que se transferiría la carga útil del paquete.
- Enviar los bytes que conformaban el mensaje mediante la línea MOSI.
- Finalizar la transferencia desactivando la señal CSN.
- Generar un pulso en la señal CE para iniciar el proceso de transmisión del paquete.

Durante esta secuencia se utilizó el analizador lógico para capturar la comunicación SPI y observar tanto los comandos enviados como los datos transferidos. Luego, se utilizó el decodificador del software para examinar la secuencia de bytes y la duración de las operaciones.

5. Análisis del alcance del sistema:

Para el análisis de alcance se planteó inicialmente la determinación del alcance teórico del sistema para las diferentes configuraciones de potencia de transmisión y sensibilidad del receptor.

Luego, se realizó una prueba de campo para comparar el comportamiento del enlace a diferentes distancias. Para cada posición se verificó la posibilidad de establecer correctamente la comunicación entre el transmisor y el receptor.

La guía establece que los alcances teóricos deben compararse posteriormente con los obtenidos mediante pruebas de campo fuera del laboratorio.

6. Análisis de errores de transmisión:

Para estudiar el comportamiento de los errores se prepararon archivos de texto con diferentes tipos de información. Se planteó una primera prueba utilizando archivos que contienen alfabetos y una segunda utilizando datos pseudoaleatorios.

Los archivos fueron transmitidos mediante el sistema inalámbrico y posteriormente se comparó la información enviada con la información recibida. A partir de esta comparación se podrá determinar el comportamiento de la tasa de error de bits.

Esta sección se desarrollará siguiendo las dos pruebas de transmisión establecidas en la guía.

7. Análisis de la señal mediante el analizador de espectros:

Finalmente, se utilizó un analizador de espectros para observar la señal inalámbrica generada por el NRF24L01.

Inicialmente se configuró el analizador en el modo de análisis de espectro y se estableció la frecuencia central de acuerdo con el canal seleccionado en el módulo. Luego se ajustaron parámetros como el span y el ancho de banda de resolución para facilitar la observación de la señal.

A partir de las mediciones se planteó verificar:

La frecuencia de operación del canal configurado.

El ancho de banda ocupado por la comunicación.

La ubicación de la señal dentro de la banda de 2,4 GHz.

La separación necesaria entre canales para evitar traslapes.

El comportamiento de la señal mediante la función de espectrograma.

También se contempla el análisis de la canalización utilizada por Wi-Fi en la banda de 2,4 GHz mediante capturas de espectrograma, de acuerdo con las actividades indicadas en la guía.

V. ANÁLISIS DE RESULTADOS

1. Verificación de la transmisión inalámbrica mediante el analizador de espectros:

Para comprobar el funcionamiento del sistema de comunicación inalámbrica, se utilizó un analizador de espectros *Keysight FieldFox*, con el propósito de observar la señal de radiofrecuencia generada por el módulo NRF24L01 durante la transmisión de mensajes. Para realizar esta prueba, se configuró el sistema en modo de transmisión continua mediante la opción LOOP, que permite enviar repetidamente el mensaje seleccionado.

En el analizador se configuró una frecuencia central de 2,471 GHz, un rango de frecuencia (span) de 1 MHz y un ancho de banda de resolución (RBW) de 7,5 kHz. También se utilizó la función Hold para conservar la traza de la señal y facilitar su observación durante las transmisiones, en la figura 4 se observa el resultado obtenido en el analizador de espectro.



Figura 4. Resultado de la medición en el analizador de espectro.

En la figura 4 se observa un pico pronunciado por encima del nivel de ruido, ubicado aproximadamente en 2,470 GHz, de acuerdo con la lectura del marcador del instrumento. Este pico corresponde a la señal de radiofrecuencia detectada durante la prueba. Además, la traza acumulada permitió observar la presencia de la señal mientras el sistema se encontraba configurado para transmitir mensajes repetidamente.

La persistencia del pico durante la prueba se relaciona con el uso del modo LOOP, ya que el programa ejecutaba repetidamente la función de transmisión con una pausa de 1 ms entre iteraciones. Es importante tener en cuenta que el NRF24L01 transmite paquetes de datos y no una portadora continua; por ello, la apariencia persistente de la señal en la pantalla se debe interpretar considerando el comportamiento repetitivo de la transmisión y el uso de la función Hold del analizador.

De acuerdo con la configuración del programa, el canal inicial del NRF24L01 era el 71, cuya frecuencia nominal se calcula mediante:

$$f_{RF} = 2400 + RF_{CH}$$
$$f_{RF} = 2400 + 71 = 2471\ MHz$$

2. Análisis de la comunicación SPI:

2.1. Lectura del registro RF_CH:

Para analizar la comunicación entre la Raspberry Pi Pico y el módulo NRF24L01 se desarrolló un programa en MicroPython que permite leer periódicamente el registro RF_CH, encargado de almacenar el canal de radiofrecuencia configurado en el transceptor. Para observar esta operación se utilizó Logic 2, con el cual se capturaron las señales de la interfaz SPI y se identificaron los bytes transmitidos y recibidos.

La interfaz SPI se configuró con una frecuencia de reloj de 1 MHz, polaridad igual a cero y fase igual a cero. Se utilizaron los pines GP18 para SCK, GP19 para MOSI, GP16 para MISO y GP20 para CSN. El pin GP17 se asignó a la señal CE del NRF24L01.

El programa ejecutó repetidamente la función *leer_registro(0x05)*, correspondiente a la lectura del registro RF_CH, con un intervalo de 500 ms entre operaciones. De esta manera, fue posible capturar varias lecturas consecutivas y seleccionar una de ellas para estudiar su secuencia de comunicación.

Código utilizado:

A continuación, se muestra el pseudocódigo utilizado:

```
INICIO

    Configurar SPI:
        Frecuencia = 1 MHz
        Polaridad = 0
        Fase = 0
        SCK = GP18
        MOSI = GP19
        MISO = GP16

    Configurar CE = GP17
    Configurar CSN = GP20

    Establecer CE = 0
    Establecer CSN = 1

    DEFINIR función leer_registro(registro):

        Establecer CSN = 0
        Enviar dirección del registro por SPI
        Recibir un byte por SPI
        Establecer CSN = 1

        Retornar el byte recibido

    FIN FUNCIÓN

    MIENTRAS el programa esté en ejecución:

        Leer el registro RF_CH (dirección 0x05)
        Mostrar el valor en decimal y hexadecimal
        Esperar 500 ms

    FIN MIENTRAS

FIN
```

Resultado en Logic 2:

En el software Logic 2 se configuró el decodificador SPI para visualizar las señales SCK, MOSI, MISO y CSN. La captura ampliada permite distinguir dos grupos de pulsos de reloj asociados al envío del comando de lectura y a la recepción del contenido del registro.

En la figura 5 se muestra la captura del resultado en Logic 2, donde se identifican los bytes transmitidos y recibidos durante la operación.

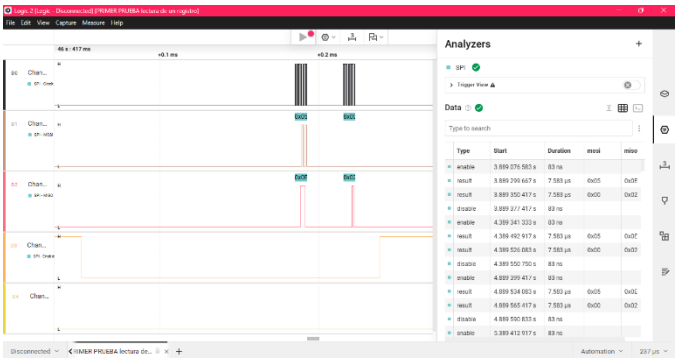


Figura 5. Captura de Logic 2 correspondiente a la lectura del registro RF_CH, donde se identifican los bytes transmitidos y recibidos durante la operación.

En la primera parte de la operación, la Raspberry Pi Pico envía el byte 0x05 por la línea MOSI. Este valor corresponde a la dirección del registro RF_CH y permite solicitar su contenido. Durante el envío del comando, el NRF24L01 devuelve por MISO el byte 0x0E, que corresponde al registro de estado STATUS.
Luego, el microcontrolador envía el byte 0x00 para generar los pulsos de reloj necesarios para recibir la información solicitada. Durante esta segunda transferencia, la línea MISO presenta el valor 0x02, que corresponde al contenido leído del registro.
Los bytes identificados en la captura se resumen en la tabla 6.

Byte	MOSI (transmitido)	MISO (recibido)	Interpretación
1	0x05	0x0E	Solicitud de lectura del registro RF_CH y respuesta de estado STATUS.
2	0x00	0x02	Generación de los pulsos de reloj y recepción del contenido del registro.

Tabla 6. Bytes identificados en la lectura del registro.

Análisis de los tiempos de transferencia:
De acuerdo con la información presentada por el decodificador de Logic 2, cada byte tuvo una duración aproximada de 7,583 microsegundos. Este valor se puede comparar con el tiempo teórico correspondiente a la transferencia de ocho bits a una frecuencia de reloj de 1 MHz.
El tiempo teórico de transferencia de un byte se calcula mediante la siguiente expresión:

$$T_b = \frac{número\ de\ bits}{frecuencia\ del\ reloj}$$

$$T_b = \frac{8\ bits}{1Ms/s} = 8\mu s$$

La duración experimental registrada, de aproximadamente 7,583 microsegundos, es cercana al valor teórico de 8 microsegundos. La diferencia puede estar relacionada con la resolución temporal de la captura y la ubicación de los marcadores utilizados para medir la transferencia.
En la captura también se observa un intervalo de habilitación de aproximadamente 83 nanosegundos en el decodificador. Este dato corresponde a la duración indicada por Logic 2 para el evento de selección y no debe confundirse con el tiempo completo de transmisión de un byte.

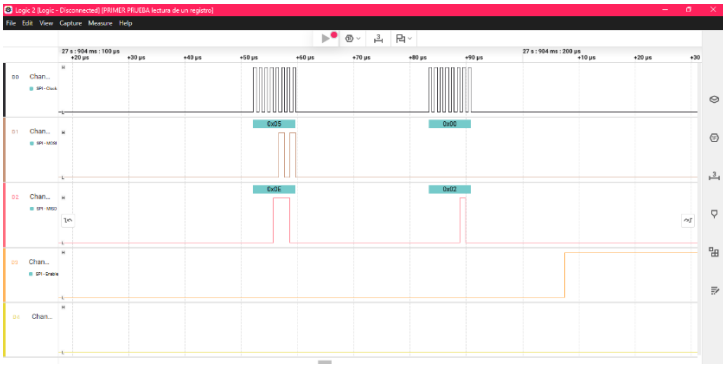


Figura 6. Pulsos de reloj y la secuencia de bytes de la operación de lectura.

A partir de la figura 6, se identificó que el valor leído del registro RF_CH fue 0x02, equivalente a 2 en decimal. Esto indica que, durante esta prueba, el NRF24L01 tenía configurado el canal 2. De acuerdo con la relación entre el número de canal y la frecuencia nominal de operación, esta configuración corresponde a 2402 MHz.
El resultado también permite comprobar que la Raspberry Pi Pico logró realizar la lectura del registro mediante SPI. Esto se evidencia en la secuencia de bytes capturada: el microcontrolador envió el comando 0x05 para solicitar la lectura del registro y, luego, transmitió 0x00 para generar los pulsos de reloj necesarios para recibir el dato. Por su parte, el NRF24L01 respondió con 0x0E, correspondiente al registro de estado STATUS, y luego entregó el valor 0x02 solicitado.
Es importante aclarar que esta lectura corresponde a la prueba inicial realizada con el código específico de lectura del registro. Por lo tanto, el canal identificado no debe confundirse con el canal 76 utilizado posteriormente en la configuración principal del sistema.
En conclusión, la captura permitió verificar el intercambio de información entre el microcontrolador y el transceptor, reconocer la estructura de una operación de lectura SPI e identificar el contenido del registro consultado. De esta manera, se comprobó el funcionamiento de la interfaz utilizada para acceder a los registros internos del NRF24L01 y configurar sus parámetros de comunicación.

2.2. Escritura del registro RF_CH:
Para esta prueba se desarrolló un programa en MicroPython con el objetivo de modificar el canal de radiofrecuencia del módulo NRF24L01 mediante la interfaz SPI. Para ello, se utilizó la función *escribir_registro(registro, valor)*, encargada de enviar el comando de escritura y el valor que se desea almacenar en el registro seleccionado.
La interfaz SPI se configuró con una frecuencia de reloj de 1 MHz, polaridad igual a cero y fase igual a cero. Se utilizaron los pines GP18 para SCK, GP19 para MOSI, GP16 para MISO, GP20 para CSN y GP17 para CE.
En esta prueba se seleccionó el registro RF_CH, cuya dirección es 0x05, y se escribió el valor decimal 76, equivalente a 0x4C en hexadecimal. Después de realizar la escritura, se ejecutó una lectura del mismo registro para comprobar el valor almacenado.

Código utilizado:
INICIO

Configurar SPI:
Frecuencia = 1 MHz
Polaridad = 0
Fase = 0
SCK = GP18
MOSI = GP19
MISO = GP16

Configurar CE = GP17
Configurar CSN = GP20

Establecer CE = 0
Establecer CSN = 1

DEFINIR función escribir_registro(registro, valor):

Establecer CSN = 0

Enviar comando (0x20 OR registro) por SPI
Enviar valor por SPI
Establecer CSN = 1

FIN FUNCIÓN

DEFINIR función leer_registro(registro):

Establecer CSN = 0
Enviar dirección del registro por SPI
Recibir un byte por SPI
Establecer CSN = 1

Retornar el byte recibido

FIN FUNCIÓN

Esperar 500 ms

Escribir 76 en el registro RF_CH (0x05)

Esperar 100 ms

Leer el registro RF_CH (0x05)
Mostrar el valor leído en decimal y hexadecimal

FIN

Análisis del resultado en Logic 2:

En la figura 7, correspondiente a la captura de Logic 2 se observan las señales SCK, MOSI, MISO y CSN durante las operaciones de escritura y lectura del registro. El decodificador SPI permite identificar los bytes intercambiados en cada transferencia.

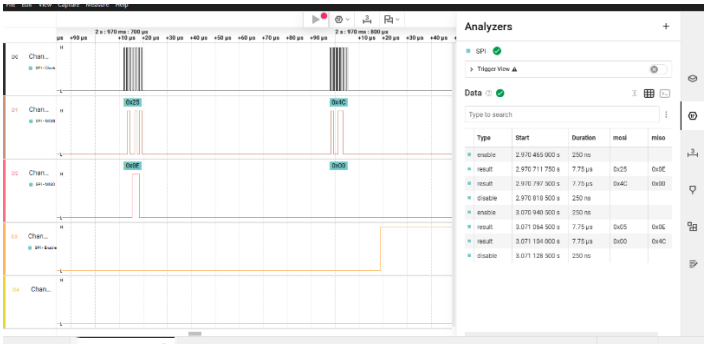


Figura 7. Resultado de la escritura de registro en Logic 2.

Durante la escritura, el microcontrolador envía los siguientes bytes:

- 0x25: comando de escritura del registro RF_CH.
- 0x4C: valor que se desea almacenar en el registro.

El comando 0x25 se obtiene al combinar el comando de escritura 0x20 con la dirección 0x05 del registro RF_CH. Por su parte, 0x4C corresponde al valor decimal 76.

La tabla 7 resume los datos identificados en la captura:

Byte	MOSI (transmitido)	MISO (recibido)	Interpretación
1	0x25	0x0E	Comando de escritura del registro RF_CH y respuesta de estado STATUS
2	0x4C	0x00	Valor enviado para configurar el canal y respuesta observada en MISO.

Tabla 7. Bytes identificados en la escritura de registro.

En la figura 8, se puede observar con mayor detalle la secuencia de pulsos de reloj y las transferencias realizadas durante la prueba. En ella se distinguen las operaciones correspondientes a la escritura del registro y a su lectura posterior.

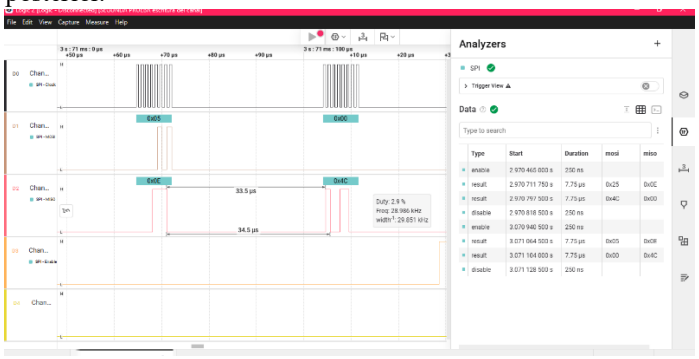


Figura 8. Verificación del registro después de la escritura en Logic 2.

Después de enviar el comando de escritura 0x25 y el valor 0x4C, el programa ejecuta una nueva operación de lectura del registro RF_CH. En esta segunda operación, el microcontrolador envía 0x05 por MOSI y recibe 0x0E por MISO durante el envío del comando. A continuación, envía 0x00 para generar los pulsos de reloj necesarios y recibe 0x4C como contenido del registro.

Los datos se resumen en la tabla 8:

Operación	MOSI (transmitido)	MISO (recibido)	Interpretación
Lectura del registro	0x05	0x0E	Solicitud de lectura y respuesta de estado
Lectura del contenido	0x00	0x4C	Valor leído del registro RF_CH

Tabla 8. Bytes transmitidos y recibidos durante la verificación del registro RF_CH.

Análisis de los tiempos de transferencia:

De acuerdo con los datos mostrados por el decodificador de Logic 2, la duración de cada byte fue de aproximadamente 7,75 µs. Este valor se puede comparar con el tiempo teórico de transmisión de ocho bits a una frecuencia de reloj de 1 MHz, que es de 8µs.

La duración experimental de 7,75 µs es cercana al valor teórico de 8 µs. Esta comparación permite verificar que los tiempos de transferencia observados son coherentes con la frecuencia de reloj configurada para la interfaz SPI.

En la segunda captura también se muestran intervalos aproximados de 33,5 µs y 34,5 µs entre determinadas transiciones de la señal MISO. Estos intervalos corresponden a las mediciones realizadas con los marcadores sobre la forma de onda y no representan la duración individual de un byte.

Finalmente, se observó que la lectura posterior a la escritura permitió identificar el valor 0x4C en el registro RF_CH, equivalente a 76 en decimal. Esto indica que el valor enviado durante la operación de escritura quedó almacenado en el registro consultado.

La frecuencia nominal asociada al canal 76 se calcula;

$$f = 2400 + 76 = 2476\,MHz.$$

Por lo tanto, la prueba permitió comprobar mediante SPI la escritura del registro RF_CH y la lectura posterior de su contenido. Además, las capturas muestran cómo el microcontrolador envía primero el comando de escritura y el dato correspondiente, y luego realiza una lectura para verificar el valor almacenado.
Esta prueba demuestra la utilidad de SPI para modificar y consultar los registros internos del NRF24L01, permitiendo configurar los parámetros necesarios para el funcionamiento del sistema de comunicación inalámbrica.

2.3. Transferencia de un paquete al NRF24L01:

Para esta prueba se desarrolló un programa en MicroPython con el objetivo de observar cómo la Raspberry Pi Pico envía los comandos y los datos de un mensaje al módulo NRF24L01 mediante la interfaz SPI.
La interfaz SPI se configuró con una frecuencia de reloj de 1 MHz, polaridad igual a cero y fase igual a cero. Se utilizaron los pines GP18 para SCK, GP19 para MOSI, GP16 para MISO, GP20 para CSN y GP17 para CE.
El mensaje seleccionado para esta prueba fue "EDIEM VALERO". El programa convirtió el texto en una secuencia de bytes mediante la codificación UTF-8 y completó la carga útil con bytes de valor cero hasta alcanzar los 32 bytes. Este procedimiento se realizó debido a que el código se diseñó para preparar una carga útil de longitud fija.

Código utilizado:

```
INICIO

  Configurar SPI:
    Frecuencia = 1 MHz
    Polaridad = 0
    Fase = 0

  Configurar pines:
    SCK = GP18
    MOSI = GP19
    MISO = GP16
    CE = GP17
    CSN = GP20

  Definir comandos:
    W_TX_PAYLOAD = 0xA0
    FLUSH_TX = 0xE1

  DEFINIR función cargar_paquete(mensaje):

    Convertir mensaje a bytes

    Completar la carga útil con bytes 0x00
    hasta alcanzar 32 bytes

    Seleccionar NRF24L01 mediante CSN = 0
    Enviar comando FLUSH_TX (0xE1)
    Finalizar operación mediante CSN = 1

    Seleccionar NRF24L01 mediante CSN = 0
    Enviar comando W_TX_PAYLOAD (0xA0)
    Enviar los 32 bytes del mensaje
    Finalizar operación mediante CSN = 1

  FIN FUNCIÓN

  MIENTRAS el programa esté en ejecución:

    Cargar el mensaje "EDIEM VALERO"
    Mostrar mensaje de confirmación en consola
    Esperar 500 ms

  FIN MIENTRAS

FIN
```

Análisis del resultado en Logic 2:

En la figura 9, se observan las señales SCK, MOSI, MISO y CSN durante la ejecución repetida de la función cargar_paquete(). El decodificador SPI permitió identificar los comandos enviados al transceptor y los bytes correspondientes al mensaje.

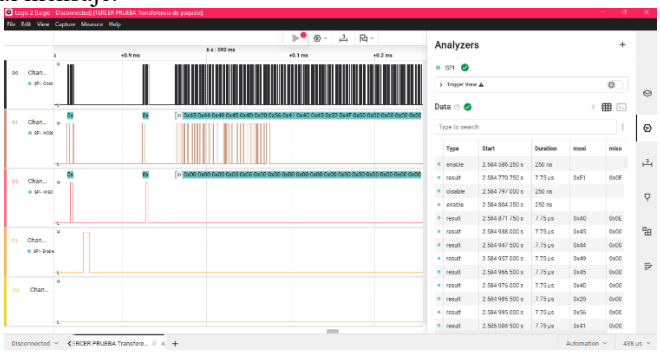


Figura 9. Transferencia de un paquete en Logic 2.

La operación comienza con el comando 0xE1, denominado FLUSH_TX, que se utiliza para limpiar la cola FIFO de transmisión del NRF24L01. Después de finalizar esta operación, el programa selecciona nuevamente el módulo mediante CSN y envía el comando 0xA0, denominado W_TX_PAYLOAD, que indica que los siguientes bytes corresponden a la carga útil que se desea almacenar en la cola de transmisión.
La secuencia de comandos observada se resume en la tabla 8.

Comando	Valor hexadecimal	Función
FLUSH_TX	0xE1	Limpia la cola FIFO de transmisión.
W_TX_PAYLOAD	0Xa1	Indica que se enviará la carga útil del paquete.

Tabla 9. Comandos SPI utilizados para preparar el paquete de transmisión.

Durante el envío de estos comandos, el decodificador muestra el byte 0x0E por la línea MISO, correspondiente al estado STATUS del transceptor. Luego, durante la transferencia de la carga útil, se observan principalmente respuestas de valor 0x00 en MISO.

Después del comando W_TX_PAYLOAD, el programa envía los bytes correspondientes al mensaje "EDIEM VALERO". Cada carácter se representa mediante un código hexadecimal según su codificación ASCII.
Los primeros 12 bytes corresponden al contenido original del mensaje, como se presenta en la tabla 10.

Carácter	Código hexadecimal
E	0x45
D	0x44
I	0x49
E	0x45
M	0x4D
Espacio	0x20
V	0x56
A	0x41
L	0x4C
E	0x45
R	0x52
O	0x4F

Tabla 10. Representación hexadecimal del mensaje transmitido por SPI.

El mensaje contiene 12 bytes. Como el programa completa la carga útil hasta 32 bytes, se agregan 20 bytes con valor 0x00 después del último carácter. Por tanto, la estructura de la carga útil queda conformada por 12 bytes de información y 20 bytes de relleno.

En la figura 9 se observa que los bytes enviados por MOSI coinciden con la representación hexadecimal del mensaje. Después de los caracteres aparecen los valores 0x00, que corresponden al relleno añadido por el programa.

Esta secuencia permite comprobar cómo un mensaje de texto se transforma en datos digitales antes de ser almacenado en el transceptor.

Análisis de los tiempos de transferencia:

En la tabla de datos de Logic 2 se observa que la duración de cada byte transferido es de aproximadamente 7,75 µs. Al comparar ambos valores, se observa que el tiempo experimental de 7,75 µs es cercano al valor teórico de 8 µs.

La carga útil contiene 32 bytes, por lo que el tiempo teórico dedicado exclusivamente a transferir esos datos es:

$$Tiempo\ de\ carga\ útil\ =\ 32\ bytes\ \times\ 8\ \mu s/byte.$$

$$Tiempo\ de\ carga\ útil\ =\ 256\ \mu s.$$

Este valor no incluye el tiempo de los comandos, los cambios de estado de CSN ni otros intervalos entre operaciones. Por ello, corresponde únicamente al tiempo teórico necesario para transferir los 32 bytes de datos.

Finalmente, la captura obtenida permite identificar las operaciones utilizadas para preparar una carga útil en el NRF24L01. Primero, el programa envía el comando FLUSH_TX para limpiar la cola de transmisión y, posteriormente, utiliza W_TX_PAYLOAD para indicar que se transferirá un nuevo paquete.

Los bytes observados en MOSI corresponden al mensaje "EDIEM VALERO", seguido de los bytes de relleno necesarios para completar los 32 bytes de la carga útil. Esto demuestra cómo el programa convierte el mensaje de texto en una secuencia de datos que puede ser procesada por el transceptor mediante SPI.

Además, la duración experimental de aproximadamente 7,75 µs por byte es cercana al tiempo teórico de 8 µs para una frecuencia de reloj de 1 MHz, lo que resulta coherente con la configuración de la interfaz utilizada durante la prueba.

En conclusión, esta prueba permitió observar la secuencia de comandos SPI, identificar la representación hexadecimal del mensaje y analizar el tiempo de transferencia de los datos. De esta manera, se complementan las pruebas de lectura y escritura de registros, mostrando otra de las operaciones necesarias para controlar el transceptor NRF24L01.

VI. CONCLUSIONES

- Mediante las pruebas realizadas se logró analizar la comunicación entre la Raspberry Pi Pico y el módulo NRF24L01 utilizando el protocolo SPI. Con el analizador lógico fue posible identificar los comandos de lectura y escritura de registros, así como la transferencia de los bytes correspondientes a una carga útil.
- La lectura del registro RF_CH permitió identificar el valor almacenado en el transceptor durante la primera prueba. Luego, mediante la escritura del valor hexadecimal 0x4C y la lectura del registro, se verificó la configuración del canal 76, cuya frecuencia nominal corresponde a 2476 MHz. Esto permitió comprobar el uso de SPI para consultar y modificar los parámetros internos del módulo.
- En la prueba de transferencia de paquetes se identificaron los comandos FLUSH_TX y W_TX_PAYLOAD, utilizados para preparar la cola de transmisión y cargar el mensaje seleccionado. Además, se observó la representación hexadecimal de los caracteres del mensaje "EDIEM VALERO" y el relleno con bytes de valor cero hasta completar los 32 bytes de la carga útil.
- Las mediciones realizadas con Logic 2 mostraron tiempos aproximados de 7,583 µs y 7,75 µs por byte en las diferentes pruebas. Estos valores son cercanos al tiempo teórico de 8 µs para una transferencia de ocho bits a una frecuencia de reloj de 1 MHz, lo que permitió relacionar las mediciones experimentales con la configuración de la interfaz SPI.
- En general, las actividades desarrolladas permitieron relacionar la programación del microcontrolador con las señales digitales observadas mediante el analizador lógico y con la señal de radiofrecuencia detectada experimentalmente. Estos resultados constituyen una base para continuar con las pruebas de alcance, errores de transmisión y caracterización espectral planteadas en la guía de laboratorio.

VII. BIBLIOGRAFÍA

[1] Murata Manufacturing, "Basics of Wireless Communication (1)," [En línea]. Disponible en: https://article.murata.com/es/article/basics-of-wireless-communication-1. [Último acceso: 8 de octubre de 2026].

[2] Descubre Arduino, "NRF24L01: Descripción del módulo," [En línea]. Disponible en: https://descubrearduino.com/nrf24l01/#Descripcion_del_nRF24L01. [Último acceso: 8 de octubre de 2026].

[3] Nordic Semiconductor, "nRF24L01+ Product Specification," septiembre de 2008. [En línea]. Disponible en: https://cdn.sparkfun.com/assets/3/d/8/5/1/nRF24L01P_Product_Specification_1_0.pdf. [Último acceso: 8 de octubre de 2026].

[4] S. Campbell, "Basics of the SPI Communication Protocol," Circuit Basics, [En línea]. Disponible en:

https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/. [Último acceso: 8 de octubre de 2026].
[5] L. Llamas, “Comunicación inalámbrica a 2,4 GHz con Arduino y NRF24L01,” Luis Llamas, [En línea]. Disponible en: https://www.luisllamas.es/comunicacion-inalambrica-a-2-4ghz-con-arduino-y-nrf24l01/. [Último acceso: 8 de octubre de 2026].
[6] Uelectronics, “Analizador lógico USB de 24 MHz y 8 canales,” [En línea]. Disponible en: https://uelectronics.com/producto/analizador-logico-usb-24mhz-8-canales/. [Último acceso: 8 de octubre de 2026].
[7] Keysight Technologies, “Learn: Información y recursos técnicos,” [En línea]. Disponible en: https://www.keysight.com/es/en/learn.html?srtby=MOST_POPULAR&contenttypes=product%2Binformation. [Último acceso: 8 de octubre de 2026].

VIII. ANEXOS

Github Paula Quintero:
https://github.com/paulaquintero2811/Laboratorio-NRF24L01
Github Ediem Valero:
https://github.com/estediemvalero-coder/Laboratorio-NRF24L01